All you need to know about Prometheus

Alexis Carbillet

August 3rd 2023

Contents

Introduction **iii**

1 Basics **1**
1.1 Exporter 1
1.2 Data 2
1.3 Literals 4
1.4 Selectors 6
1.5 Configuration 8
1.6 Promtool 10
1.7 Records 11
1.8 Storage 12
1.9 Gotchas 14

2 Operators **17**
2.1 Aggregators 17
2.2 Binary 19
2.3 Matching 22
2.4 Precedence 24

3 Functions **25**
3.1 Alert 25
3.2 Histogram 27
3.3 Label 29
3.4 Range 31
3.5 Time-shifting 33

4 API **35**
4.1 Querying 35
4.2 Results 38
4.3 Service Discovery 41
4.4 Management 42

5 To go further **45**
5.1 Install Prometheus in Linux environment 45
5.2 Create a personalized exporter 47
5.3 Complex requests with node-exporter 49
5.4 Create a personalized alert 51
5.5 Good practices 53

6 Conclusion **55**

7 About the author **57**

Introduction

Prometheus is an open-source software tool used for monitoring and alerting in computer systems and applications. It helps track the health and performance of different components in a system, such as servers, databases, and applications, by collecting metrics or data about their behavior.

Imagine you have a car and you want to know how well it's performing. You might want to measure things like the speed, fuel consumption, or engine temperature. Prometheus does something similar for computer systems. It keeps an eye on various aspects of a system, like its CPU usage, memory usage, or response time. It collects this information over time and stores it in a database.

Prometheus also allows you to set up rules and conditions to trigger alerts when something goes wrong. For example, if the CPU usage goes above a certain threshold or if the response time of a website becomes too slow, Prometheus can send an alert to let you know that there's a problem.

In summary, Prometheus is a tool that monitors and keeps track of the performance and health of computer systems by collecting data, storing it, and notifying you when something goes wrong. It helps ensure that your systems are running smoothly and allows you to take action when necessary.

Chapter 1

Basics

1.1 Exporter

In the context of Prometheus, a Prometheus exporter is a software component or module that collects metrics from a specific system or application and makes them available for Prometheus to scrape and store.

Let's say you have a system or application that you want to monitor with Prometheus. However, Prometheus doesn't know how to directly collect metrics from that system or application. This is where the Prometheus exporter comes into play. The exporter acts as a bridge between the system or application you want to monitor and Prometheus.

The exporter is responsible for gathering relevant metrics from the system or application and exposing them in a format that Prometheus can understand. It typically provides a well-defined endpoint or interface where Prometheus can send HTTP requests to collect the metrics. The exporter converts the internal metrics into a standardized format, usually using text-based formats like plain text, JSON, or Protocol Buffers.

Once the Prometheus exporter is set up and running, Prometheus periodically scrapes the exporter's endpoint to fetch the metrics. It collects the metrics, stores them in its database, and allows you to visualize and analyze them using Prometheus's querying and alerting capabilities.

Prometheus exporters are available for a wide range of systems and applications, including web servers, databases, cloud platforms, messaging systems, and more. They enable you to monitor and track the performance of these systems and applications alongside other metrics collected by Prometheus.

In summary, a Prometheus exporter is a software component that collects metrics from a specific system or application, converts them into a format Prometheus can understand, and exposes them through an endpoint for Prometheus to scrape and store. It enables Prometheus to monitor various systems and applications by gathering their metrics in a standardized way.

1.2 Data

Prometheus, an open-source monitoring and alerting toolkit, primarily uses a single data type called "metric" to represent and store data. The metric data type in Prometheus is a time series data structure consisting of a unique name, a set of key-value pairs known as labels, and a timestamped value.

Prometheus supports four main types of metrics:

1. Counter: A counter is a monotonically increasing value that represents a cumulative count or a rate of events. Counters can only increase and are typically used for tracking the number of occurrences of a specific event.

2. Gauge: A gauge represents a value that can both increase and decrease over time. It is used for capturing instantaneous values such as the current system CPU usage or memory consumption.

3. Histogram: A histogram samples observations (usually durations or values) and counts them in configurable buckets. It provides statistical information about the distribution of values, such as the average, quantiles, and sum of observed values.

4. Summary: A summary is similar to a histogram but calculates quantiles on the fly. It provides a sliding time window for calculating the quantiles, making it more suitable for long-term monitoring.

These metric types allow Prometheus to collect and store different kinds of data, enabling monitoring and analysis of various aspects of systems and applications.

1. Counter:

```
http_requests_total{method="GET", status="200"} 1024
```

This example represents a counter named "http_requests_total", with labels "method" and "status". It tracks the total count of HTTP GET requests with a status code of 200, which is currently at 1024.

2. Gauge:

```
cpu_usage{instance="webserver1"} 0.75
```

Here, "cpu_usage" is a gauge metric that measures the CPU usage of the "webserver1" instance. The value 0.75 represents the current CPU usage, which could be a percentage between 0 and 1 or a normalized value.

3. Histogram:

```
http_request_duration_seconds_bucket{method="POST", status="200",
    le="0.1"} 1500
```

This histogram metric, http_request_duration_seconds_bucket, tracks the number of HTTP POST requests with a status code of 200 that fall into the bucket of duration less than or equal to 0.1 seconds. In this case, the bucket has recorded 1500 occurrences.

4. Summary:

```
http_request_latency_seconds{quantile="0.95"} 0.037
```

A summary metric named "http_request_latency_seconds" calculates quantiles on the fly. This example represents the 95th percentile latency of HTTP requests, which is currently at 0.037 seconds.

These examples demonstrate how metrics are structured in Prometheus, with metric names, labels, and corresponding values to represent different aspects of system monitoring and analysis.

1.3 Literals

In Prometheus, literals refer to the constant values that can be used in queries and alerting rules. Here are some common literals used in Prometheus:

1. Numeric Literals: These include integers and floating-point numbers. For example:

- 42: An integer literal.
- 3.14: A floating-point literal.

2. String Literals: These are enclosed in double quotes (”). For example:

- ”hello”: A string literal containing the text ”hello”.

3. Boolean Literals: These represent the truth values of true and false.

4. Duration Literals: These represent time durations. They can be expressed using a combination of numbers and units. For example:

- 5s: A duration of 5 seconds.
- 2h30m: A duration of 2 hours and 30 minutes.

5. Time Literals: These represent specific points in time and are formatted using the ISO 8601 standard. For example:

- 2023-07-02T12:34:56Z: A time literal representing July 2, 2023, at 12:34:56 UTC.

6. Regular Expression Literals: These are enclosed in slashes (/). They are used for pattern matching in regular expression-based queries and alerting rules. For example:

- /ĥttp/: A regular expression literal that matches strings starting with ”http”.

7. Range Vector Selector Literals: These are used to select a range of time series data. They use the square bracket notation. For example:

- my_metric[5m]: A range vector selector literal that selects data for the metric ”my_metric” over the past 5 minutes.

These are some of the common literals used in Prometheus. They provide the necessary flexibility to construct queries and alerting rules based on different types of data and time periods.

Here are examples of Prometheus queries for each type of literal:

1. Numeric Literal:

- Query:

```
http_requests_total > 100
```

- Description: This query checks if the metric http_requests_total is greater than 100.

2. String Literals:

- Query:

```
http_response_code{code="200"}
```

- Description: This query selects time series where the http_response_code metric has the label "code" with a value of "200".

3. Boolean Literal:

- Query:

```
up == 1
```

- Description: This query selects time series where the "up" metric has a value of 1, indicating that the target is up and running.

4. Duration Literal:

- Query:

```
rate(my_metric[5m])
```

- Description: This query calculates the per-second rate of the metric "my_metric" over the past 5 minutes.

5. Time Literal:

- Query:

```
http_requests_total{instance="webserver1"}[1h]
```

- Description: This query selects the time series for http_requests_total from the instance labeled "webserver1" over the past 1 hour.

6. Regular Expression Literal:

- Query:

```
count(count by (status) (http_response_code =~ /^5\d{2}$/))
```

- Description: This query counts the occurrences of HTTP response codes starting with "5" (e.g., 500, 503, etc.).

7. Range Vector Selector Literal:

- Query:

```
avg_over_time(http_requests_total[5m])
```

- Description: This query calculates the average value of http_requests_total over the past 5 minutes for each time series.

1.4 Selectors

Prometheus, an open-source monitoring and alerting toolkit, provides various time series selectors for querying and retrieving metrics data. Here are some of the commonly used time series selectors in Prometheus:

1. Metric name selector: You can select time series data based on the metric name. For example, "up" selects all time series with the metric name "up".

2. Label matching selector: Prometheus uses labels to identify and filter time series data. You can use label matching selectors to specify conditions for label values. The commonly used label matching operators are "=", "!=", "= " (regex match), and "! " (regex non-match). For example, 'job="node"' selects all time series with the label "job" having the value "node".

3. Regular expression selector: You can use regular expressions to match metric names or label values. For example, '{__name__= "http_.*_requests_total"}' selects all time series with metric names starting with 'http_' and ending with '_requests_total'.

4. Aggregation selector: Prometheus allows aggregation of time series using functions such as sum, avg, min, max, etc. Aggregation selectors enable you to perform calculations on multiple time series and retrieve a single result. For example, "sum(http_requests_total)" calculates the sum of all time series with the metric name "http_requests_total".

5. Offset modifier: You can apply the offset modifier to a selector to shift the time range for retrieving time series data. It allows you to select time series data from a specific time window relative to the current evaluation time. For example, "http_requests_total offset 5m" selects all time series with the metric name "http_requests_total" from the past 5 minutes.

6. Range vector selector: Prometheus supports range vectors, which allow you to select a range of time series data over a specified time interval. Range vector selectors are used in combination with functions like rate, increase, delta, etc., to calculate rates or changes over time. For example, "rate(http_requests_total[5m])" calculates the per-second rate of increase of the http_requests_total metric over the past 5 minutes.

These are some of the common time series selectors in Prometheus. They provide flexibility in querying and analyzing metrics data to gain insights and create alerts based on specific criteria.

Here are examples for each of the time series selectors in Prometheus:

1. Metric name selector:

- Query:

```
up
```

- Description: Selects all time series with the metric name "up".

2. Label matching selector:

- Query:

```
up{job="node"}
```

- Description: Selects all time series with the label "job" having the value "node".

3. Regular expression selector:

- Query:

```
{__name__=~"http_.*_requests_total"}
```

- Description: Selects all time series with metric names starting with "http_" and ending with "_requests_total".

4. Aggregation selector:

- Query:

```
sum(http_requests_total)
```

- Description: Calculates the sum of all time series with the metric name http_requests_total.

5. Offset modifier:

- Query:

```
http_requests_total offset 5m
```

- Description: Selects all time series with the metric name http_requests_total from the past 5 minutes.

6. Range vector selector:

- Query:

```
rate(http_requests_total[5m])
```

- Description: Calculates the per-second rate of increase of the http_requests_total metric over the past 5 minutes.

These examples demonstrate the usage of each time series selector in Prometheus. Please note that the queries provided are for illustrative purposes and may need to be adapted to your specific use case and metric names in your Prometheus setup.

1.5 Configuration

Configuring Prometheus involves setting up its configuration file to define targets, scraping intervals, alerting rules, and more. Prometheus uses a YAML-based configuration format. Here are the basic steps and options you can configure:

1. Create Configuration File: Start by creating a configuration file named prometheus.yml or any name you prefer.

2. Global Configuration:

```
global:
    scrape_interval: 15s  # How often to scrape targets
```

3. Scrape Configuration: Define the targets to be scraped. These can be HTTP endpoints exposed by your services.

```
scrape_configs:
    - job_name: 'example-job'
        static_configs:
        - targets: ['localhost:9090']  # Replace with your service
    endpoints
```

4. Alerting Rules: Define alerting rules for triggering alerts based on metric conditions.

```
rule_files:
    - 'alert.rules.yml'  # Separate file for alerting rules
```

5. Alertmanager Configuration: Configure how alerts are managed and notified.

```
alerting:
    alertmanagers:
        - static_configs:
            - targets: ['localhost:9093']  # Replace with
    Alertmanager endpoint
```

6. Retain and Prune Data: Configure data retention and pruning.

```
storage:
    retention: 30d  # How long to retain data
    tsdb:
        max_block_duration: 2h
```

7. Service Discovery: Prometheus supports various service discovery mechanisms like Kubernetes, Consul, etc.

8. Remote Write / Remote Read: Optionally, configure Prometheus to write or read data from remote storage or other Prometheus instances.

9. TLS Configuration: For secure communication, you can configure TLS settings.

10. Scrape Configurations: Customize scraping settings for specific jobs or targets.

Remember, these are just basic options. You can configure many more advanced settings and integrations depending on your use case. After editing the configuration file, restart Prometheus for changes to take effect.

Prometheus also provides a web UI which you can access at http://localhost:9090 (default port) to explore your configured targets, metrics, and alerts.

1.6 Promtool

promtool is a command-line utility that comes bundled with Prometheus. It is used for various purposes related to Prometheus configuration and rule file validation. promtool can be used to perform syntax checking, linting, and validation of Prometheus configuration files, as well as alerting and recording rules.

Here's how you can use promtool and some of its advantages and disadvantages:

Usage:

1. Syntax Checking: You can use

```
promtool check config your_config_file
```

to validate the syntax and correctness of your Prometheus configuration file.

2. Rule Validation:

```
promtool check rules your_rule_file
```

allows you to validate alerting and recording rule files.

Advantages:

1. Configuration Validation: promtool helps catch syntax errors and potential issues in your Prometheus configuration and rule files before applying them to a running Prometheus instance. This reduces the chances of configuration-related errors causing problems in your monitoring setup.

2. Rule Verification: It ensures that your alerting and recording rules are correctly defined and conform to Prometheus' rule syntax. This helps in maintaining accurate and reliable alerting mechanisms.

3. Early Detection of Issues: By validating your configuration and rules early in the development or deployment process, you can catch issues before they impact your monitoring and alerting setup.

4. Consistency: promtool enforces consistent formatting and adherence to Prometheus' configuration and rule file syntax, which is especially useful in collaborative or team environments.

Disadvantages:

1. Command Line Interface: Some users may find the command-line interface of promtool less intuitive than graphical user interfaces or other validation tools.

2. Complexity: For users who are new to Prometheus or YAML-based configuration, understanding and diagnosing certain errors might be challenging.

3. Limited Features: While promtool is great for configuration and rule validation, it doesn't cover all aspects of Prometheus administration, such as runtime metric analysis, data visualization, or advanced debugging.

In summary, promtool is a valuable tool for ensuring the correctness and reliability of your Prometheus configuration and rule files. It helps catch potential errors early in the development or deployment process, contributing to a more stable and effective monitoring setup. However, it's important to also consider using additional tools and practices for comprehensive monitoring, alerting, and visualization needs.

1.7 Records

Recording rules in Prometheus are used to precompute and store new time series based on existing ones. They are a way to create new metrics by applying functions or expressions to the existing metrics, making complex queries more efficient and enabling the creation of aggregated or transformed metrics that can be used for visualization, alerting, and analysis.

Recording rules are defined in the Prometheus configuration file (prometheus.yml) using the recording_rules section. Here's an example of how recording rules are defined:

```
rule_files:
    - 'recording_rules.yml'
```

In the recording_rules.yml file, you define the recording rules using PromQL (Prometheus Query Language). Each rule consists of a name, an expression, and an optional labels section. Here's an example:

```
groups:
    - name: my_recording_rules
        rules:
        - record: http_requests_total_sum
            expr: sum(http_requests_total) by (job)
```

In this example, a new metric named http_requests_total_sum is created by summing the http_requests_total metric grouped by the job label. This can be useful for visualizing the total number of HTTP requests per job.

Recording rules are especially beneficial in scenarios where certain queries or aggregations are computationally expensive and frequently used. By precomputing these aggregations, you can reduce the load on the Prometheus server and make querying more efficient.

When defining recording rules, keep the following considerations in mind:

1. Efficiency: Recording rules should be used for aggregations or calculations that are frequently used in queries. This helps improve query performance.

2. Cardinality: Be cautious when creating recording rules that can generate a high cardinality of time series, as it can impact Prometheus' memory usage.

3. Maintenance: Regularly review and update your recording rules to ensure they remain relevant to your monitoring needs.

4. Testing: Before deploying new recording rules, test them in a non-production environment to ensure they behave as expected.

5. Documentation: Maintain clear documentation for your recording rules to ensure other team members understand their purpose and usage.

Recording rules are a powerful feature of Prometheus that allow you to customize and optimize your monitoring setup to better suit your organization's specific requirements.

1.8 Storage

1. Prometheus Server Configuration:

- Configure Prometheus to scrape targets and collect metrics. This is typically done through the prometheus.yml configuration file.
- Define the scrape_configs section to specify the targets to scrape.
- Set up any additional configuration options, like alerting rules, service discovery, and retention policies.

2. Metrics Collection and Storage:

- Prometheus scrapes metrics from configured targets at regular intervals.
- It stores these metrics locally in its time-series database.
- Prometheus uses its own storage format optimized for fast querying and efficient storage.

3. Data Retention and Pruning:

- Configure the data retention period in the Prometheus configuration (storage.retention option).
- This setting determines how long the data is retained in the storage.
- Older data is automatically pruned based on the configured retention period.

4. Querying Metrics:

- Prometheus provides APIs (such as the Query API and Range Query API) to query and retrieve metrics.
- You can use PromQL (Prometheus Query Language) to express complex queries to fetch the required data.

5. Visualization and Alerting:

- Prometheus can be used with visualization tools like Grafana to create dashboards and visualize the collected metrics.
- You can also configure alerting rules in Prometheus to trigger alerts based on metric conditions.

Prometheus' internal storage system is well-optimized for its use case of monitoring and alerting. However, there are also options to use remote storage integrations if you have specific requirements. For instance:

- Remote Storage: Prometheus supports remote read and write APIs, allowing you to use other storage systems for long-term retention. You can configure Prometheus to offload data to remote storage systems like Thanos, Cortex, or any other system that supports the Prometheus remote read/write API.

- Prometheus Remote Write: This feature allows you to send metrics to remote storage systems in real-time. You can configure Prometheus to send metrics to various backends, including cloud-based storage solutions or other monitoring platforms.

When choosing a storage strategy, consider factors like data retention requirements, query performance, scalability, and integration with existing monitoring infrastructure.

For most use cases, using Prometheus' built-in storage is a straightforward and effective approach. However, if you need extended retention or more advanced features, you might explore integrations with remote storage solutions.

1.9 Gotchas

Prometheus is a powerful open-source monitoring and alerting toolkit widely used in the field of DevOps and system monitoring. While Prometheus offers numerous benefits, there are some potential challenges and gotchas to be aware of:

1. Data Storage: Prometheus follows a pull-based model where it scrapes metrics from various targets. It stores the scraped data in a time-series database format. However, Prometheus is designed to store data for a limited duration by default, typically a few weeks. If long-term data retention is required, you'll need to set up additional mechanisms like remote storage or integrate with other systems.

2. Cardinality and Label Usage: Prometheus uses labels to identify and differentiate time series data. While labels provide flexibility and granularity, excessive label usage or high cardinality can lead to increased resource consumption and performance issues. It's important to carefully choose and manage labels to maintain efficient and effective monitoring.

3. Alerting and Alert Management: Prometheus has a built-in alerting mechanism that allows you to define and trigger alerts based on specific conditions. However, managing alerts at scale can be challenging. It's crucial to establish effective alerting strategies, handle alert deduplication, and ensure proper routing and escalation mechanisms are in place.

4. Scalability: As the number of monitored targets and metrics increases, the resource requirements for Prometheus can grow significantly. Storage, memory, and CPU usage may become bottlenecks if not appropriately scaled. It's important to monitor Prometheus itself and plan for horizontal scaling or sharding if necessary.

5. Service Discovery: Prometheus relies on service discovery mechanisms to identify and scrape metrics from targets. While it provides integrations with popular service discovery tools, configuring and managing service discovery correctly can be complex. Ensuring proper configuration and addressing potential issues related to dynamic environments is crucial for reliable monitoring.

6. Data Exporting and Integration: While Prometheus has a powerful querying language (PromQL) and supports various visualization options, you might need to export data for integration with other systems or downstream processing. Exporting data from Prometheus requires additional components such as exporters or exporters-specific integrations, and this integration setup should be considered while designing the monitoring architecture.

7. Backup and Disaster Recovery: As Prometheus stores data locally, it's important to have proper backup and disaster recovery mechanisms in place to prevent data loss in case of failures or incidents. Regularly backing up the Prometheus data directory and having a recovery plan will help ensure data integrity and availability.

It's important to thoroughly understand these gotchas and considerations when implementing Prometheus to make the most of its capabilities and address potential challenges effectively. The Prometheus documentation and community

resources can provide further guidance on these topics.

Here are examples to illustrate each of the gotchas with Prometheus:

1. Data Storage: Let's say you have Prometheus set up to store data for a duration of two weeks. However, you suddenly realize that you need to analyze historical data spanning several months. Without a proper long-term data storage solution or integration with external systems like object storage or data lakes, accessing and analyzing data beyond the default retention period becomes challenging.

2. Cardinality and Label Usage: Suppose you have a Prometheus instance monitoring a Kubernetes cluster with hundreds of pods. You decide to label each time series with various labels like pod name, namespace, and labels associated with the application. Over time, the number of unique label combinations (cardinality) increases significantly, potentially consuming excessive resources and impacting performance. Careful label selection and avoiding unnecessary label proliferation is important to maintain optimal performance.

3. Alerting and Alert Management: You have defined multiple alerts in Prometheus to monitor the health and performance of your application. However, due to improper alert configuration or lack of deduplication, you start receiving excessive alerts for the same issue, leading to alert fatigue. It's crucial to establish proper alerting strategies, deduplicate alerts, and implement mechanisms for intelligent routing and escalation to avoid overwhelming teams with redundant alerts.

4. Scalability: As your infrastructure grows and the number of targets and metrics monitored by Prometheus increases, you notice that the Prometheus server is struggling to handle the load. Queries become slower, and resource usage spikes. It becomes evident that the current Prometheus instance needs to be scaled horizontally by deploying additional instances or shards to distribute the load effectively and maintain performance.

5. Service Discovery: You have set up Prometheus with Kubernetes service discovery to automatically discover and scrape metrics from pods. However, you encounter issues where new pods are not being discovered or scraped due to misconfigured service discovery settings or intermittent connectivity issues. Ensuring proper service discovery configuration and addressing dynamic environment challenges is crucial to maintaining accurate monitoring.

6. Data Exporting and Integration: You want to export data from Prometheus to an external data analysis platform for more advanced analytics and reporting. However, you realize that Prometheus doesn't have a direct integration with your chosen platform. To address this, you need to set up an exporter or use a specific integration tool to export Prometheus data in a compatible format and establish the necessary pipeline for integration.

7. Backup and Disaster Recovery: A critical hardware failure occurs, leading to the loss of data stored in Prometheus. Without a proper backup strategy in place, you face challenges in recovering the lost data. Regularly backing up the Prometheus data directory and having a well-defined disaster recovery plan would have allowed you to quickly restore the data and minimize the impact of such incidents.

These examples highlight potential challenges and the importance of addressing them appropriately while working with Prometheus.

Chapter 2

Operators

2.1 Aggregators

Prometheus provides several aggregation operators that allow you to perform calculations and aggregations on the collected metrics. Here are some commonly used aggregation operators in Prometheus:

1. sum: Calculates the sum of the selected metric over a given time range.

2. avg: Computes the average value of the selected metric over a specified time period.

3. min: Returns the minimum value of the selected metric within a given time range.

4. max: Returns the maximum value of the selected metric within a given time range.

5. count: Counts the number of times the selected metric has been recorded within a given time frame.

6. quantile: Computes the quantile value of the selected metric over a specified time period. For example, "quantile(0.95)" will give you the 95th percentile value.

7. rate: Calculates the per-second average rate of increase of the selected metric over a given time window.

8. irate: Similar to "rate", but provides an instant rate of increase at each point in time.

9. delta: Computes the difference between the first and last value of the selected metric within a given time range.

10. stddev: Calculates the standard deviation of the selected metric over a specified time period.

These operators can be used in Prometheus's query language, PromQL, to perform aggregations and calculations on your metrics data. By combining these operators with various functions and modifiers, you can derive meaningful insights and create custom monitoring and alerting rules based on your specific requirements.

Here are examples of each aggregation operator using PromQL syntax:

1. sum:

```
sum(metric_name)
```

2. avg:

```
avg(metric_name)
```

3. min:

```
min(metric_name)
```

4. max:

```
max(metric_name)
```

5. count:

```
count(metric_name)
```

6. quantile:

```
quantile(0.95, metric_name)
```

7. rate:

```
rate(metric_name[5m])
```

8. irate:

```
irate(metric_name[1m])
```

9. delta:

```
delta(metric_name[1h])
```

10. stddev:

```
stddev(metric_name)
```

Note that in these examples, "metric_name" should be replaced with the actual name of the metric you want to aggregate. The time ranges within square brackets [ ] specify the duration over which the aggregation is performed (e.g., 5m for 5 minutes, 1h for 1 hour).

Keep in mind that PromQL is a flexible query language, and you can combine these aggregation operators with other PromQL functions and modifiers to create more complex queries and calculations based on your specific needs.

2.2 Binary

While PromQL does not have traditional binary operators, it supports a range of functions and operators for manipulating and aggregating data. Here are some key operators in PromQL:

- Arithmetic Operators:
  - Addition: +
  - Subtraction: -
  - Multiplication: *
  - Division: /
  - Modulo: %
- Comparison Operators:
  - Equal to: ==
  - Not equal to: !=
  - Greater than: >
  - Less than: <
  - Greater than or equal to: >=
  - Less than or equal to: <=
- Logical Operators:
  - AND: and
  - OR: or
  - NOT: unless or not
- Set Operators:
  - Union: |
  - Intersection: &
  - Difference: \

These operators can be used to construct queries in PromQL to filter, aggregate, and perform calculations on the time-series data stored in Prometheus. It's worth noting that PromQL is specifically designed for working with time-series data, and its operators and functions are optimized for that purpose.

1. Arithmetic Operators:

- Addition: "metric1 + metric2"

  Example:

```
http_requests_total + http_errors_total
```

- Subtraction: "metric1 - metric2"

  Example:

```
cpu_usage - cpu_idle
```

- Multiplication: "metric1 * metric2"

  Example:

```
price * quantity
```

- Division: "metric1 / metric2"

  Example:

```
response_time / request_count
```

- Modulo: "metric1 % metric2"

  Example:

```
total_requests % 100
```

2. Comparison Operators:

- Equal to: "metric1 == metric2"

  Example:

```
status_code == 200
```

- Not equal to: "metric1 != metric2"

  Example:

```
error_code != 0
```

- Greater than: "metric1 > metric2"

  Example:

```
cpu_usage > 80
```

- Less than: "metric1 < metric2"

  Example:

```
response_time < 100
```

- Greater than or equal to: "metric1 >= metric2"

  Example:

```
response_time $>$= 200
```

- Less than or equal to: "metric1 <= metric2"

  Example:

```
error_count <= 5
```

3. Logical Operators:

- AND: "metric1 and metric2"

  Example:

```
cpu_usage > 80 and memory_usage > 70
```

- OR: "metric1 or metric2"

  Example:

```
status_code == 500 or status_code == 503
```

- NOT: "unless(metric)"

  Example:

```
unless(cpu_usage > 90)
```

4. Set Operators:

- Union: "metric1 — metric2"

  Example:

```
http_requests_total{status="200"} | http_requests_total{status
    ="500"}
```

- Intersection: "metric1 & metric2"

  Example:

```
http_requests_total{status="200"} & http_requests_total{method
    ="GET"}
```

- Difference: "metric1 metric2"

  Example:

```
http_requests_total{status="200"} \ http_requests_total{method
    ="POST"}
```

These examples demonstrate how the operators can be used in PromQL queries to perform arithmetic calculations, compare metrics, combine conditions, and manipulate sets of data.

2.3 Matching

Prometheus, an open-source monitoring and alerting system, provides several matching operators to filter and aggregate time series data. These operators are used in Prometheus's query language, PromQL (Prometheus Query Language), to define the selection criteria for metrics. Here are the commonly used matching operators in Prometheus:

1. Equality (=): Matches time series with labels that have the exact specified value.

2. Inequality (!=): Matches time series with labels that do not have the specified value.

3. Regular expression (=): Matches time series with labels that match the specified regular expression.

4. Negated regular expression (!): Matches time series with labels that do not match the specified regular expression.

5. Set membership ({labelname}): Matches time series that have a specific label name, regardless of its value.

6. Range matching ({labelname="value1"-"value2"}): Matches time series with labels that have values falling within the specified range.

7. Aggregation (sum, avg, min, max, etc.): Allows aggregating multiple time series into a single result using various aggregation functions.

These operators can be combined to create complex queries for selecting, filtering, and aggregating time series data in Prometheus. They provide flexibility in defining the desired metrics and conditions to monitor and analyze your system.

Here are examples for each of the matching operators in Prometheus:

1. Equality (=):

```
http_requests_total{method="GET"}
```

It matches the time series with the label "method" having the value "GET". This example selects all the time series representing HTTP GET requests.

2. Inequality (!=):

```
http_requests_total{status!="200"}
```

It matches the time series with the label "status" not equal to "200". This example selects all the time series representing HTTP requests with a status other than 200.

3. Regular expression (=):

```
http_requests_total{path=~"/api/v[0-9]+/.*"}
```

It matches the time series with the label "path" matching the regular expression "/api/v[0-9]+/.*". This example selects all the time series representing HTTP requests with paths starting with "/api/v" followed by a version number.

4. Negated regular expression (!):

```
http_requests_total{path!~"/admin/.*"}
```

It matches the time series with the label "path" not matching the regular expression "/admin/.*". This example selects all the time series representing HTTP requests excluding those with paths starting with "/admin/".

5. Set membership ({labelname}):

```
{job="prometheus",instance="localhost:9090"}
```

It matches the time series with the labels "job" and "instance" having the specified values. This example selects all the time series belonging to the Prometheus server running on localhost:9090.

6. Range matching ({labelname="value1"-"value2"}):

```
http_requests_total{response_time="0.1-1.0"}
```

It matches the time series with the label "response_time" falling within the range from 0.1 to 1.0. This example selects all the time series representing HTTP requests with response times between 0.1 and 1.0 seconds.

7. Aggregation (sum, avg, min, max, etc.):

```
sum(http_requests_total) by (job)
```

It aggregates all the time series representing HTTP requests, summing them up based on the "job" label. This example calculates the total number of HTTP requests per job.

These examples illustrate how each matching operator can be used in PromQL queries to filter and aggregate time series data in Prometheus.

2.4 Precedence

In Prometheus there are no specific precedence operators like those found in programming languages. However, Prometheus Query Language (PromQL) does have a specific order of evaluation for its functions and operators. The order of evaluation is as follows:

1. Parentheses: Expressions inside parentheses are evaluated first.

2. Unary Operators: Unary operators like "-" (negation) and "+" (identity) are evaluated next.

3. Multiplication and Division: The "*" (multiplication) and "/" (division) operators are evaluated from left to right.

4. Addition and Subtraction: The "+" (addition) and "-" (subtraction) operators are evaluated from left to right.

5. Comparison Operators: Comparison operators like "==" (equal), "!=" (not equal), "¡" (less than), "¿" (greater than), "¡=" (less than or equal to), and "¿=" (greater than or equal to) are evaluated next.

6. Logical Operators: Logical operators like "and", "or", and "unless" are evaluated from left to right.

It's important to note that Prometheus evaluates expressions strictly from left to right, without any operator precedence rules to override the default order.

Here's an example of a PromQL expression to illustrate the order of evaluation:

```
sum(rate(http_requests_total{job="myapp"}[5m])) / 60 * 100
```

In this example, the expression is evaluated by summing the results of the subquery:

```
rate(http_requests_total{job="myapp"}[5m])
```

Then the result is divided by 60 and finally multiplied by 100.

Remember that Prometheus evaluates expressions in a linear manner, so if you need to enforce specific operator precedence, you can use parentheses to group subexpressions and control the order of evaluation.

Chapter 3

Functions

3.1 Alert

In Prometheus, alert functions are used to define conditions and rules for generating alerts based on the monitored metrics. When the conditions specified by these functions are met, Prometheus triggers an alert that can be used to notify system administrators or take automated actions. Here are some commonly used alert functions in Prometheus:

1. ALERT(): This function is used to define an alert rule. It takes two arguments: the name of the alert and the expression that evaluates to a boolean value. When the expression evaluates to true, the alert is fired.

2. label_join(): This function is used to concatenate label values together into a new label. It takes two arguments: the destination label name and a delimiter. This function is often used to create a new label that provides additional context for an alert.

3. absent(): This function checks whether a specific time series is absent or missing. It takes one argument: the expression that represents the time series. If the time series does not exist or has no samples within the specified time range, the function evaluates to true and triggers the alert.

4. changes(): This function counts the number of times a time series changes its value within a specified time range. It takes one argument: the expression that represents the time series. The function triggers the alert if the number of changes exceeds a specified threshold.

5. increase(): This function calculates the rate of increase of a time series over a specified time range. It takes two arguments: the expression that represents the time series and the duration over which the increase is calculated. The function triggers the alert if the increase exceeds a specified threshold.

6. rate(): This function calculates the average rate of change of a time series over a specified time range. It takes one argument: the expression that represents the time series. The function triggers the alert if the rate exceeds a specified threshold.

7. up(): This function checks the availability of a target by querying the "up" metric, which indicates whether a scrape target is up or down. It takes no arguments and evaluates to 1 if the target is up and 0 if it's down. This function is often used to create alerts for target availability.

These are just a few examples of the alert functions available in Prometheus. The alerting functionality in Prometheus is highly customizable, allowing you to define complex conditions and actions based on your specific monitoring requirements.

Here are examples for each of the alert functions in Prometheus:

1. ALERT(): Define an alert rule named "HighCPUUsage" that triggers when the CPU usage exceeds 80% for more than 5 minutes:

```
ALERT HighCPUUsage
IF avg(cpu_usage) > 0.8
FOR 5m
```

2. label_join(): Create a new label named "alert_info" by joining the "instance" and "severity" labels with a hyphen ("-") delimiter:

```
label_join(alert_info, "-", instance, severity)
```

3. absent(): Trigger an alert if the metric "http_requests_total" is absent for the past 5 minutes:

```
ALERT MissingHttpRequests
IF absent(http_requests_total) FOR 5m
```

4. changes(): Generate an alert if the value of the metric "errors_total" changes more than 10 times within the last 10 minutes:

```
ALERT HighErrorFluctuations
IF changes(errors_total) > 10
FOR 10m
```

5. increase(): Raise an alert if the rate of increase of the metric "request_latency_seconds" is above 0.05 over the last 1 hour:

```
ALERT HighLatencyIncrease
IF increase(request_latency_seconds[1h]) > 0.05
```

6. rate(): Trigger an alert if the average rate of change of the metric "http_requests_total" exceeds 100 requests per second over the last 5 minutes:

```
ALERT HighRequestRate
IF rate(http_requests_total[5m]) > 100
```

7. up(): Create an alert if a target is down by checking the availability of the "up" metric:

```
ALERT TargetDown
IF up() == 0
```

These examples demonstrate how the alert functions can be used to define specific conditions and generate alerts based on the monitored metrics in Prometheus. Remember to adapt these examples to your own metric names and threshold values according to your monitoring setup.

3.2 Histogram

In Prometheus, histograms are a type of metric used to track the distribution of values over a given range. Histograms are often used to measure the latency or response time of an operation and provide insights into the distribution of those values. PromQL provides several histogram-specific functions that can be used to analyze and manipulate histogram metrics. Here's an explanation of each histogram function:

1. histogram_quantile(ϕ, histogram_vector): This function calculates the ϕ-quantile (where ϕ is a value between 0 and 1) of the given histogram vector. It returns the value below which the given fraction of observations in the histogram falls. For example, "histogram_quantile(0.95, my_histogram)" will calculate the 95th percentile value of the histogram.

2. histogram_metric_bucket: When you define a histogram metric in Prometheus, it automatically generates multiple time series, each representing a specific bucket or range of values. The generated time series have the suffix_bucket. You can query these time series to access the count of observations falling into specific value ranges.

3. histogram_metric_sum: This function returns the sum of all observed values in a histogram metric. It provides the total sum of the values across all buckets.

4. histogram_metric_count: This function returns the total count of observations in a histogram metric. It represents the number of values that have been recorded across all buckets.

5. histogram_metric_bucket(le): This function returns the count of observations in a histogram metric that are less than or equal to the given value le. It is useful for querying the count of values falling into specific buckets or value ranges.

These functions allow you to perform various calculations and analysis on histogram metrics in Prometheus. You can retrieve specific quantiles, access individual buckets, obtain the sum and count of observations, and perform other operations to gain insights into the distribution of values captured by the histogram.

These examples demonstrate how you can use each histogram function in Prometheus to query and analyze histogram metrics:

1. histogram_quantile(ϕ, histogram_vector):

```
histogram_quantile(0.95, my\_histogram)
```

This example calculates the 95th percentile value of the my_histogram metric.

2. histogram_metric_bucket:

```
my_histogram_bucket{le="0.1"}
```

This example retrieves the count of observations falling into the bucket with an upper bound of 0.1 in the "my_histogram" metric.

3. histogram_metric_sum:

```
my_histogram_sum
```

This example returns the sum of all observed values in the "my_histogram" metric

4. histogram_metric_count:

```
my_histogram_count
```

This example returns the total count of observations in the "my_histogram" metric.

5. histogram_metric_bucket("le"):

```
my_histogram_bucket{le="0.5"}
```

This example retrieves the count of observations falling into the bucket with an upper bound of 0.5 in the my_histogram metric.

3.3 Label

In Prometheus, labels are key-value pairs associated with time series data. They allow you to add metadata and context to your metrics, enabling more granular and specific querying. Prometheus provides various label functions to manipulate and filter time series data based on these labels. Here are the main label functions in Prometheus:

1. label_replace: This function allows you to replace or modify labels of a time series. It takes four arguments: the input vector, the target label name, a regular expression to match the source label value, and a replacement string.

2. label_join: With this function, you can concatenate label values into a new label. It takes two arguments: the input vector and a delimiter string to join the labels.

3. label_drop: This function removes specified labels from the input vector, effectively reducing the dimensionality of the data.

4. label_keep: The "label_keep" function filters out all labels from the input vector except the specified ones. It helps to focus on specific labels while discarding the rest.

5. label_match: This function selects time series that match a given regular expression pattern for a specific label. It takes two arguments: the input vector and the regular expression pattern.

6. label_replace_all: Similar to "label_replace", this function replaces or modifies multiple labels of a time series in one step. It takes three arguments: the input vector, a set of pairs of regular expressions and replacement strings, and an optional parameter to control the behavior of replacing multiple labels.

7. label_map: This function maps source label values to target label values based on a provided mapping. It takes three arguments: the input vector, a mapping of source to target labels, and an optional default value for unmatched source labels.

8. label_set: With this function, you can add or modify label values of a time series. It takes three arguments: the input vector, the label name to set or modify, and the label value to assign.

These label functions provide powerful capabilities for manipulating and filtering time series data based on their labels in Prometheus. They allow you to modify, transform, or select specific time series based on the metadata associated with them, enabling more targeted and flexible querying and analysis.

These examples demonstrate how each label function can be used in PromQL to manipulate and transform time series data based on their labels:

1. label_replace:

```
label_replace(my_metric{job="job1"}, "new_label", "$1", "source_
    label", "(.*)")
```

This example replaces the value of the "new_label" with the value of "source_label" for the "my_metric" time series. The regular expression "(.*)" captures the value of "source_label" and assigns it to "new_label".

2. label_join:

```
label_join(my_metric{job="job1"}, "new_label", ",", "label1", "
    label2")
```

This example joins the values of "label1" and "label2" with a comma (",") as the delimiter and assigns the result to "new_label" for the "my_metric" time series.

3. label_drop:

```
label_drop(my_metric{job="job1"}, "label1", "label2")
```

This example removes the "label1" and "label2" labels from the "my_metric" time series, reducing its dimensionality.

4. label_keep:

```
label_keep(my_metric{job="job1"}, "label1", "label2")
```

This example keeps only the "label1" and "label2" labels for the "my_metric" time series and discards all other labels.

5. label_match:

```
label_match(my_metric{job="job1"}, "label_name", "regex_pattern")
```

This example selects time series from the "my_metric" metric that have a "label_name" matching the specified "regex_pattern".

6. label_replace_all:

```
label_replace_all(my_metric{job="job1"}, {
  source_label1=~"regex_pattern1" => "replacement1",
  source_label2=~"regex_pattern2" => "replacement2"
})
```

This example replaces multiple labels ("source_label1" and "source_label2") with their corresponding replacement values for the "my_metric" time series, based on the provided regular expression patterns.

7. label_map:

```
label_map(my_metric{job="job1"}, { "source_label1" => "target_
    label1", "source_label2" => "target_label2" }, "default_value")
```

This example maps the values of "source_label1" and "source_label2" to their respective target labels ("target_label1" and "target_label2") for the "my_metric" time series. If a source label value is not present in the mapping, it uses the specified default value.

8. label_set:

```
label_set(my_metric{job="job1"}, "new_label", "new_value")
```

This example adds or modifies the label new_label to have the value new_value for the my_metric time series.

3.4 Range

Prometheus provides several range functions that are used to specify time ranges for querying data. These functions help define the temporal scope of the query. Here are the main range functions in Prometheus:

1. time():

- This function returns the current server time as the timestamp. It is commonly used to refer to the current time in queries.

  Example:

```
time()
```

2. timestamp():

- This function returns the timestamp of a given point in time. It takes a date-time string as an argument and returns the corresponding timestamp.

  Example:

```
timestamp("2023-07-02T10:30:00Z")
```

3. offset():

- The offset() function shifts a time series by a given duration. It is useful for comparing data at different time intervals or for comparing data at the same time interval but at different points in time.

  Example:

```
offset(metric_name[2h])
```

4. start():

- This function returns the start time of a time range. It is typically used in combination with range functions to specify the beginning of a time interval.

  Example:

```
start()
```

5. end():

- The end() function returns the end time of a time range. It is often used in combination with range functions to specify the end of a time interval.

  Example:

```
end()
```

6. s():

- The s() function is used to specify a duration in seconds. It is used to define the length of a time range in terms of seconds.

  Example:

```
metric_name[s]
```

7. m():

- This function specifies a duration in minutes. It is used to define the length of a time range in terms of minutes.

  Example:

```
metric_name[m]
```

8. h():

- The h() function defines a duration in hours. It is used to specify the length of a time range in terms of hours. Example:

```
metric_name[h]
```

9. d():

- This function specifies a duration in days. It is used to define the length of a time range in terms of days. Example:

```
metric_name[d]
```

These range functions help you define the time window for querying data in Prometheus. By combining these functions with other PromQL functions, you can effectively analyze and extract insights from time series metrics.

3.5 Time-shifting

Prometheus provides a few time-shifting functions that allow you to shift or offset time series data. These functions help you analyze metrics at different points in time or compare data from different time periods. Here are the main time-shifting functions in PromQL:

1. offset(): The offset() function shifts a time series by a given duration. It takes two parameters: the duration to shift and the input time series. The resulting time series will have its data shifted by the specified duration. This can be useful for comparing data at different time intervals or aligning data from different sources. For example, "offset(5m, my_metric)" shifts the "my_metric" time series 5 minutes into the future.

2. timestamp(): The timestamp() function returns the timestamp of a given point in time. It takes a single parameter, which can be a scalar value or a range vector. The function returns the timestamp corresponding to the provided input. This is useful for extracting specific timestamps from time series data. For example, timestamp(1625200000) returns the timestamp corresponding to the Unix time 1625200000.

These time-shifting functions enable you to manipulate and analyze time series data in Prometheus. They offer flexibility in comparing and aligning metrics at different points in time, extracting specific timestamps, or working with the current time in your queries.

Here are examples of each time-shifting function in Prometheus:

1. offset():

Suppose you have a metric called request_latency representing the latency of incoming requests. To compare the latency values 10 minutes ago with the current values, you can use the offset() function:

```
request_latency - offset(10m, request_latency)
```

This query subtracts the request_latency time series shifted by 10 minutes from the current request_latency time series, giving you the difference in latency between the two time periods.

2. timestamp():

Assume you have a metric called http_requests_total, which represents the total number of HTTP requests made to your server. To retrieve the timestamp of a specific data point, you can use the timestamp() function:

```
timestamp(http_requests_total{status_code="200"})
```

This query returns the timestamp corresponding to the latest data point of the http_requests_total time series with a status_code of "200". It helps you identify when a successful HTTP request occurred.

Chapter 4

API

4.1 Querying

Querying the Prometheus API allows you to retrieve and analyze time series data collected by Prometheus, a popular open-source monitoring and alerting system. The Prometheus API provides several features for querying and retrieving data. Here are some of the key features:

1. Data retrieval: The Prometheus API allows you to retrieve time series data based on specific metrics, labels, and time ranges. You can query for instantaneous values or range-based data over a specified duration.

2. Metrics and labels: Prometheus organizes data using a key-value pair structure called labels. With the API, you can query data based on specific metric names and filter results using labels. This allows you to narrow down your queries and fetch specific subsets of data.

3. PromQL: The Prometheus Query Language (PromQL) is a powerful query language used to express complex queries against the time series data. It supports a wide range of functions, operators, and aggregations, allowing you to perform calculations, filtering, and transformations on the data.

4. Range queries: The API supports range queries, allowing you to fetch data over a specified time range. You can specify the start and end time of the range, as well as the step size to control the resolution of the returned data.

5. Aggregation: Prometheus provides various aggregation functions that can be applied to the queried data. These include functions like sum, average, min, max, and more. Aggregation enables you to perform calculations on multiple time series or reduce the resolution of the data.

6. Sorting and ordering: The API allows you to sort and order the returned data based on specific criteria. You can specify the order in which the time series or data points are returned, making it easier to analyze and visualize the data.

7. Metadata and service discovery: In addition to time series data, the Prometheus API provides access to metadata about the metrics and labels avail-

able in the system. This includes information about metric names, label names, and their values. You can also use the API for service discovery to fetch information about the available Prometheus instances.

These are some of the essential features offered by the Prometheus API for querying and retrieving time series data. The exact details and capabilities may vary depending on the specific version of Prometheus you are using. It's recommended to consult the Prometheus API documentation or relevant resources for more detailed information.

Here are examples illustrating each of the features mentioned:

1. Data retrieval:

- Query for the instantaneous value of a metric:

```
/api/v1/query?query=<metric_name>
```

- Query for a range of data over a specific time range:

```
/api/v1/query_range?query=<metric_name>&start=<start_time>&end
    =<end_time>&step=<step_size>
```

2. Metrics and labels:

- Query for a specific metric with a label filter:

```
/api/v1/query?query=<metric_name>{<label_name>="<label_value
    >"}
```

3. PromQL:

- Query for the average value of a metric over a time range:

```
/api/v1/query_range?query=avg(<metric_name>)&start=<start_time
    >&end=<end_time>&step=<step_size>
```

4. Range queries:

- Fetch data for the last 5 minutes:

```
/api/v1/query_range?query=<metric_name>&start=<current_time-5m
    >&end=<current_time>&step=<step_size>
```

5. Aggregation:

- Calculate the maximum value of a metric over a time range:

```
/api/v1/query_range?query=max(<metric_name>)&start=<start_time
    >&end=<end_time>&step=<step_size>
```

6. Sorting and ordering:

- Sort the returned time series by their metric values in descending order:

```
/api/v1/query_range?query=<metric_name>&start=<start_time>&end
    =<end_time>&step=<step_size>&sort=-value
```

7. Metadata and service discovery:

- Fetch all available metrics:

```
/api/v1/label/__name__/values
```

- Discover Prometheus instances:

```
/api/v1/targets
```

Note that these examples provide a general idea of how the queries might look like. The actual queries may require additional parameters or formatting based on the specific Prometheus API version and configuration. It's essential to consult the Prometheus API documentation for the exact syntax and options available in your environment.

4.2 Results

The Prometheus API provides various features and functionalities to interact with the Prometheus server. Here are some key features of the Prometheus API:

1. Querying Metrics: The Prometheus API allows you to query metrics data stored in the Prometheus time-series database. You can specify metric names, labels, time ranges, and other parameters to retrieve specific metrics or aggregate data.

2. Retrieving Metrics Metadata: You can retrieve metadata information about metrics, such as their names, labels, and descriptions. This feature is helpful for discovering available metrics and understanding their meanings.

3. Data Range Queries: Prometheus API enables you to query metrics data within a specific time range. You can specify start and end timestamps or use relative time expressions to retrieve metrics data for a specific period.

4. Aggregation and Filtering: The API supports various functions for aggregating and filtering metric data. You can apply functions like sum, average, minimum, maximum, and others to calculate aggregated values or filter data based on specific conditions.

5. Alerting Rules: Prometheus has a built-in alerting system that can generate alerts based on predefined rules. The API allows you to manage alerting rules, retrieve their status, and manage silences (temporary suppression of alerts).

6. Service Discovery: Prometheus supports service discovery mechanisms to dynamically discover and monitor targets (such as servers or containers) in an automated manner. The API provides endpoints to manage service discovery configurations.

7. Runtime Configuration: You can dynamically configure Prometheus through the API. This includes modifying scrape intervals, retention periods, storage configurations, and other runtime settings.

8. Exporting and Importing Data: Prometheus API supports data export and import in various formats, including Prometheus' own text-based exposition format. This allows you to transfer metrics data between different Prometheus instances or integrate with other monitoring systems.

Here are examples of the results you might obtain when using the Prometheus API for different features:

1. Querying Metrics:

- Example query:

```
http://prometheus-server/api/v1/query?query=up
```

- Result:

```
Returns a list of instances that are currently up or running.
```

2. Retrieving Metrics Metadata:

- Example query:

```
http://prometheus-server/api/v1/metadata
```

- Result:

```
Returns metadata information about available metrics, such as
    their names, labels, and descriptions.
```

3. Data Range Queries:

- Example query:

```
http://prometheus-server/api/v1/query_range?query=cpu_usage&
    start=1625086800&end=1625090400&step=60
```

- Result:

```
Returns CPU usage data for a specific time range (from
    1625086800 to 1625090400) with a step of 60 seconds.
```

4. Aggregation and Filtering:

- Example query:

```
http://prometheus-server/api/v1/query?query=avg(request_
    latency_seconds)
```

- Result:

```
Returns the average value of the `request_latency_seconds`
    metric across all instances.
```

5. Alerting Rules:

- Example query:

```
http://prometheus-server/api/v1/alerts
```

- Result:

```
Returns the current status of all active alerts generated by
    Prometheus based on predefined rules.
```

6. Service Discovery:

- Example query:

```
http://prometheus-server/api/v1/service-discovery/targets
```

- Result:

```
Returns a list of discovered targets (e.g., servers or
    containers) that Prometheus is monitoring.
```

7. Runtime Configuration:

- Example query:

```
http://prometheus-server/api/v1/admin/runtime-config
```

- Result:

```
Returns the current runtime configuration settings of the
    Prometheus server, such as scrape intervals and retention
    periods.
```

8. Exporting and Importing Data:

- Example query:

```
http://prometheus-server/api/v1/export
```

- Result:

```
Returns the metrics data in Prometheus' text-based exposition
    format, which can be exported and used for data transfer
    or integration purposes.
```

Please note that the examples provided are based on a hypothetical Prometheus server and may not represent the actual API endpoints or query parameters. It's important to consult the Prometheus documentation or your specific Prometheus setup for accurate endpoint details and query syntax.

4.3 Service Discovery

HTTP Service Discovery (HTTP SD) is a mechanism in Prometheus that allows you to dynamically discover and monitor targets (usually endpoints of your applications or services) using simple HTTP requests. With HTTP SD, Prometheus can automatically find and scrape metrics from your services without requiring you to manually configure each target.

Here's how HTTP Service Discovery works in Prometheus:

1. Service Endpoint Exposes Metrics: Your application or service exposes its metrics via an HTTP endpoint. These metrics are typically available in a specific format like Prometheus exposition format.

2. HTTP Service Discovery Configuration: In your Prometheus configuration, you set up a service discovery configuration using the http_sd_configs section.

```
scrape_configs:
    - job_name: 'http-sd-example'
      http_sd_configs:
        - url: 'http://example.com/discover-endpoints'
```

3. Discovery Endpoint: The url parameter points to an HTTP endpoint (http://example.com/discover-endpoints in the example) that provides a list of target endpoints for Prometheus to scrape. This endpoint should respond with a JSON array containing objects describing the targets.

4. Target Information: Each object in the JSON array contains information about a target to be scraped. This includes the target's targets, labels, and optional metrics_path. Labels can be used to attach additional metadata to the target.

```
[
  {
    "targets": ["service1:9100", "service2:9100"],
    "labels": {
    "job": "node-exporter",
    "env": "production"
    }
  }
]
```

5. Scraping and Discovery: Prometheus periodically queries the specified discovery endpoint (http://example.com/discover-endpoints) and dynamically updates its list of targets based on the response. It then scrapes metrics from the discovered targets according to the configured scrape_interval.

HTTP Service Discovery simplifies the process of managing and monitoring dynamic environments where services can be added or removed frequently. It allows Prometheus to stay up to date with the changing landscape of your infrastructure without requiring manual configuration adjustments.

4.4 Management

Prometheus provides a set of management APIs that allow you to interact with and manage various aspects of the Prometheus server and its components. These APIs are useful for tasks such as querying metrics, retrieving configuration information, and managing runtime behavior. Here are some key management APIs provided by Prometheus:

1. Query API (/api/v1/query):

- This API allows you to execute instant queries to retrieve metric data.
- You can specify the metric name, labels, and time range to retrieve specific data points.
- It returns a JSON response containing the query results.

2. Range Query API (/api/v1/query_range):

- Similar to the Query API, but used for range queries over a time interval.
- Useful for retrieving time series data for graphing or analysis.
- Returns a JSON response with time series data points within the specified range.

3. Label Values API (/api/v1/label/your_label_name/values):

- Allows you to retrieve the distinct values for a given label name.
- Useful for dynamic label-based selection when querying metrics.

4. Targets API (/api/v1/targets):

- Provides information about the targets that Prometheus is configured to scrape.
- Includes details such as target's address, labels, last scrape timestamp, etc.

5. Alertmanager API (/api/v1/alerts):

- Retrieves a list of active alerts along with their labels and annotations.
- Useful for monitoring the current state of your alerting rules.

6. Configuration API (/api/v1/status/config):

- Allows you to retrieve the effective runtime configuration of Prometheus.
- Useful for validating or inspecting your Prometheus configuration.

7. Runtime Configuration Reload (/-/reload):

- This API endpoint triggers a reload of the Prometheus configuration.

- Changes made to the configuration file take effect after invoking this endpoint.

8. Metrics API (/metrics):

- Prometheus itself exposes various internal metrics about its performance and behavior through this API.
- These metrics can be scraped by another Prometheus instance or monitoring system to monitor the health of your Prometheus server.

It's important to note that these APIs are typically accessed using HTTP GET requests to the specified endpoints. Responses are usually in JSON format, making them easily consumable by scripts and applications.

Here are examples of HTTP requests for each of the Prometheus management APIs mentioned earlier. Keep in mind that you'll need to replace placeholders (such as label_name, target_id, etc.) with actual values relevant to your setup.

1. Query API (/api/v1/query):

Retrieve instant metric value for a specific metric and label combination.

```
GET /api/v1/query?query=your_metric_name{your_label_name="your_
    label_value"}
```

2. Range Query API (/api/v1/query_range):

Retrieve time series data for a specific metric and label combination within a time range.

```
GET /api/v1/query_range?query=your_metric_name{your_label_name="
    your_label_value"}&start=your_start_timestamp&end=your_end_
    timestamp&step=your_sampling_interval
```

3. Label Values API (/api/v1/label/your_label_name/values):

Retrieve distinct values for a specific label.

```
GET /api/v1/label/your_label_name/values
```

4. Targets API (/api/v1/targets):

Retrieve information about configured scrape targets.

```
GET /api/v1/targets
```

5. Alertmanager API (/api/v1/alerts):

Retrieve a list of active alerts.

```
GET /api/v1/alerts
```

6. Configuration API (/api/v1/status/config):

Retrieve the effective runtime configuration of Prometheus.

```
GET /api/v1/status/config
```

7. Runtime Configuration Reload (/-/reload):

Trigger a reload of the Prometheus configuration.

```
POST /-/reload
```

8. Metrics API (/metrics):
Retrieve internal metrics exposed by Prometheus itself.

```
GET /metrics
```

Please note that for many of these requests, you will need to replace placeholders like your_metric_name, your_label_name, your_label_value, your_start_timestamp, your_end_timestamp, your_sampling_interval, and so on with actual values.

Also, remember to access these APIs on the appropriate base URL where your Prometheus server is running. The examples shown here assume a generic base URL, so you will need to adjust them according to your Prometheus server's configuration and deployment.

Chapter 5

To go further

5.1 Install Prometheus in Linux environment

To install Prometheus in a Linux environment, you can follow these steps. I'll assume you are using Ubuntu, but the process should be similar for other Linux distributions:

1. Download Prometheus:

Visit the Prometheus download page (https://prometheus.io/download) and get the latest version. You'll want the tarball with the Linux AMD64 architecture.

2. Extract the tarball:

Use the following command to extract the downloaded tarball (replace "prometheus-x.x.x.linux-amd64.tar.gz" with the actual filename):

```
tar xvfz prometheus-x.x.x.linux-amd64.tar.gz
```

3. Move the files:

Change to the extracted directory and move the binaries to the appropriate location. We'll move the Prometheus binary to "/usr/local/bin":

```
cd prometheus-x.x.x.linux-amd64
sudo mv prometheus /usr/local/bin/
sudo mv promtool /usr/local/bin/
```

4. Create a user (optional but recommended):

It's a good practice to run Prometheus as a non-root user for security reasons. Create a new user, for example "prometheus", and assign appropriate permissions to its data directory:

```
sudo useradd -M -r -s /bin/false prometheus
sudo mkdir /var/lib/prometheus
sudo chown prometheus:prometheus /var/lib/prometheus
```

5. Configuration:

Create a configuration file for Prometheus. You can use the example configuration file provided (prometheus.yml) or create your custom one.

```
sudo vim /etc/prometheus/prometheus.yml
```

Add your desired configuration to this file. Here's a simple example that scrapes the local machine every 15 seconds:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9090']
```

Save and close the file.

6. Service setup:

To manage Prometheus as a service, create a systemd service file for it:

```
sudo vim /etc/systemd/system/prometheus.service
```

Add the following content to the file:

```
[Unit]
Description=Prometheus Server
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
    --config.file /etc/prometheus/prometheus.yml \
    --storage.tsdb.path /var/lib/prometheus \
    --web.console.templates=/usr/local/bin/consoles \
    --web.console.libraries=/usr/local/bin/console_libraries

[Install]
WantedBy=multi-user.target
```

Save and close the file.

7. Enable and start the service:

Start the Prometheus service and enable it to start on system boot:

```
sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
```

8. Access Prometheus UI:

Prometheus should now be running, and you can access its web interface by navigating to "http://localhost:9090" in your web browser.

That's it! Prometheus should now be installed and running on your Linux system. Remember to configure appropriate firewall rules and security measures to protect your Prometheus instance in a production environment.

5.2 Create a personalized exporter

Creating a personalized Prometheus exporter involves developing a custom application or script that collects metrics from a specific data source and exposes them in a format that Prometheus can scrape. To achieve this, you'll need to follow these general steps:

1. Choose a data source: Decide on the data you want to monitor and export as Prometheus metrics. This could be anything from application-specific metrics, system performance data, or data from external APIs.

2. Pick a programming language: You can use various programming languages like Python, Go, Java, or others to build your exporter. The choice of language depends on your familiarity and the requirements of your data source.

3. Include Prometheus client libraries: Prometheus provides official client libraries for different languages that simplify metric collection and exporting. You'll need to include the appropriate client library in your project.

4. Implement metric collection: Write code to collect the metrics from your data source. This could involve querying databases, gathering data from APIs, or monitoring system-level metrics.

5. Expose metrics over HTTP: Metrics should be exposed over HTTP in a format that Prometheus can scrape. The most common format is plaintext with a specific text-based format, though you can also use Protobuf or JSON.

6. Register metrics and update values: Use the Prometheus client library to register your custom metrics and update their values with the data collected in step 4.

7. Handle errors: Implement proper error handling to ensure your exporter continues to run smoothly and doesn't crash in case of issues.

8. Test your exporter: Verify that your exporter is working as expected by running it and accessing its metrics endpoint using a web browser or a tool like cURL.

9. Deploy the exporter: Deploy your exporter to the desired environment, making sure it's accessible to the Prometheus server.

Here's a basic example using Python and the "prometheus_client" library:

```
from prometheus_client import start_http_server, Gauge
import random
import time

# Create a Prometheus Gauge
custom_metric = Gauge('custom_metric', 'This is a custom metric')

def collect_custom_metric():
    # Implement your logic here to collect the data
    # For this example, we'll use a random value
    return random.random()

if __name__ == '__main__':
    # Start the HTTP server on port 8000 (default)
    start_http_server(8000)

    while True:
```

```
        # Collect the custom metric value
        value = collect_custom_metric()

        # Update the Prometheus Gauge with the collected value
        custom_metric.set(value)

        # Sleep for some time before collecting the next data point
        time.sleep(10)
```

Remember that this is a simple example, and in a real-world scenario, you would replace the "collect_custom_metric()" function with the logic to collect data from your actual data source.

After running your exporter, you can configure Prometheus to scrape the metrics from the exporter's HTTP endpoint. Add a new job to the Prometheus configuration file (usually prometheus.yml):

```
scrape_configs:
  - job_name: 'custom_metrics'
    static_configs:
      - targets: ['your_exporter_ip:8000']  # Replace with the
    actual IP of your exporter
```

Restart Prometheus to start scraping the metrics from your personalized exporter.

Keep in mind that handling authentication, encryption, and other security considerations might be necessary depending on your deployment environment.

5.3 Complex requests with node-exporter

Node-exporter is a popular tool used to collect various system-level metrics from a Linux/Unix server and expose them via HTTP for monitoring and analysis. Below are five complex requests you can make on node-exporter metrics:

1. Calculate average CPU utilization over the last hour for a specific node:

```
avg_over_time(node_cpu_seconds_total{job="node-exporter", mode="
    idle"}[1h])
```

2. Monitor and alert on unusual disk I/O patterns:

```
absent(rate(node_disk_read_bytes_total{job="node-exporter", device
    ="sda"}[5m])) OR absent(rate(node_disk_written_bytes_total{job
    ="node-exporter", device="sda"}[5m]))
```

This query checks if there's no disk I/O activity (reads or writes) on the sda disk for the last 5 minutes. If no data is available, it might indicate an issue, and an alert can be triggered.

3. Identify the top memory-consuming processes on a node:

```
topk(5, 100 - (node_memory_MemFree_bytes{job="node-exporter"} /
    node_memory_MemTotal_bytes{job="node-exporter"}) * 100)
```

This query calculates the memory usage percentage for each process by dividing used memory ("MemTotal - MemFree") by total memory ("MemTotal"). Then, it selects the top 5 processes with the highest memory usage.

4. Calculate network bandwidth utilization over a specific period for a network interface:

```
irate(node_network_receive_bytes_total{job="node-exporter", device
    ="eth0"}[1h])
```

This query calculates the incoming network bandwidth rate (bytes per second) for the "eth0" interface over the last hour using the "irate" function.

```
irate(node_network_transmit_bytes_total{job="node-exporter", device
    ="eth0"}[1h])
```

This query calculates the outgoing network bandwidth rate (bytes per second) for the "eth0" interface over the last hour using the "irate" function.

5. Monitor and visualize system load and resource utilization trends:

```
node_load1{job="node-exporter"}
```

This query retrieves the 1-minute load average for the node, which represents the average number of processes in the run queue or waiting for CPU time over the last minute.

```
100 - (avg by (instance) (irate(node_cpu_seconds_total{job="node-
    exporter", mode="idle"}[5m])) * 100)
```

This query calculates the CPU utilization percentage for each node by subtracting the average idle CPU time from 100%.

```
node_memory_MemTotal_bytes{job="node-exporter"} - node_memory_
    MemFree_bytes{job="node-exporter"} - node_memory_Buffers_bytes{
    job="node-exporter"} - node_memory_Cached_bytes{job="node-
    exporter"}
```

This query calculates the used memory by subtracting free memory, buffers, and cached memory from the total memory.

5.4 Create a personalized alert

Creating a personalized alert in Prometheus involves a few steps, including defining alerting rules and setting up alerting configurations. Prometheus is a monitoring and alerting toolkit often used with Grafana for visualization. Here's a general guide on how to create a personalized alert in Prometheus:

1. Define Alerting Rules:

Alerting rules are used to define the conditions under which an alert should be triggered. These rules are written in PromQL (Prometheus Query Language). You need to specify the metric, the condition, and the threshold that should trigger the alert. For example, let's say you want to create an alert for high CPU usage on a server. The PromQL rule might look like this:

```
# Alert rule to trigger when CPU usage crosses 80%
alert: HighCPUUsage
  expr: 100 * (1 - (avg by(instance)(irate(node_cpu_seconds_total{
    mode="idle"}[5m]))))
          > 80
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High CPU Usage on {{ $labels.instance }}"
    description: "CPU usage is {{ humanize $value }}% on {{ $labels
    .instance }}"
```

2. Save Alerting Rules:

Save the alerting rules in a file with a .rules extension, for example, "alerts.rules". You can have multiple alert rules defined in a single file.

3. Configure Prometheus:

Open the Prometheus configuration file ("prometheus.yml") and add a "rule_files" section to specify the path to your alerting rules file. The configuration should look something like this:

```
rule_files:
  - "alerts.rules"
```

4. Reload or Restart Prometheus:

After updating the Prometheus configuration file, you need to reload or restart Prometheus to apply the changes. Prometheus will now start evaluating the alerting rules and trigger alerts accordingly.

5. Configure Alertmanager:

Prometheus relies on an external component called Alertmanager to handle alerts. You need to set up Alertmanager and configure it to send alert notifications. In the Alertmanager configuration file, you can define notification receivers like email, Slack, or other integrations. For example:

```
receivers:
- name: 'email-alert'
  email_configs:
  - to: 'your-email@example.com'
    send_resolved: true
```

6. Connect Prometheus to Alertmanager:

In the Prometheus configuration file ("prometheus.yml"), add the following section to specify the Alertmanager endpoint:

```
alerting:
  alertmanagers:
  - static_configs:
    - targets: ['localhost:9093']  # The address where Alertmanager
     is running
```

7. Reload or Restart Prometheus and Alertmanager:

After making changes to the Alertmanager configuration, you need to reload or restart both Prometheus and Alertmanager to apply the configurations.

Now, when Prometheus evaluates the alerting rules and detects an alert condition, it will send the alert to Alertmanager. Alertmanager will then handle the alerts based on the configured receivers, such as sending an email or a notification to Slack.

Remember that the exact steps may vary depending on your setup and the versions of Prometheus and Alertmanager you are using. Always refer to the official documentation of Prometheus and Alertmanager for the most up-to-date and detailed instructions.

5.5 Good practices

Prometheus is a powerful monitoring and alerting tool widely used in the IT industry. To make the most of Prometheus and ensure a robust and effective monitoring system, here are some good practices to follow:

1. Define clear monitoring objectives: Clearly define what you want to monitor and set specific goals for your monitoring system. This will help you focus on collecting relevant metrics and avoid unnecessary overhead.

2. Instrument your code: Instrument your applications and services with Prometheus client libraries to expose relevant metrics. This involves adding code snippets to your applications that expose metrics like HTTP request counts, latencies, error rates, etc.

3. Choose appropriate metric types: Prometheus supports various metric types, such as counters, gauges, histograms, and summaries. Understand the differences between them and choose the most appropriate type for each metric you expose.

4. Use labels effectively: Labels in Prometheus allow you to add dimensions to your metrics, enabling better slicing and dicing of data. Use labels wisely to capture additional information about your metrics, such as instance, job, environment, etc.

5. Set up alerting rules: Define alerting rules in Prometheus to notify you when certain conditions are met. This helps you proactively respond to potential issues before they become critical.

6. Use Grafana for visualization: Grafana is a popular companion tool for Prometheus that provides rich visualization capabilities. Use Grafana to create meaningful dashboards and gain insights from your monitoring data.

7. Monitor Prometheus itself: Set up monitoring for the Prometheus server and other components to ensure that your monitoring system is running smoothly. This includes monitoring resource usage, retention, and the overall health of the Prometheus server.

8. Ensure data retention and storage: Carefully configure data retention and storage options to strike a balance between keeping historical data and managing storage requirements.

9. Use exporters for third-party integrations: Prometheus exporters are specific tools that expose metrics from various systems and applications. Utilize existing exporters or create custom ones to monitor third-party components in your infrastructure.

10. Scraping best practices: When configuring scraping jobs, ensure you set appropriate scraping intervals based on the data's volatility and the server's resource capacity. Avoid scraping unnecessary endpoints and implement proper authentication mechanisms if required.

11. Backup and restore: Regularly back up your Prometheus data and configurations to prevent data loss in case of failures or accidents.

12. Monitor for cardinality explosion: Keep an eye on the number of unique label combinations (cardinality) in your metrics. High cardinality can lead to increased storage and query costs, so it's essential to manage it effectively.

13. Upgrade Prometheus regularly: Stay up-to-date with the latest releases of Prometheus to benefit from bug fixes, improvements, and new features.

14. Document your monitoring setup: Document your Prometheus configuration, alerting rules, and any custom exporters or integrations you have set up. This documentation will be valuable for onboarding new team members and troubleshooting.

By following these good practices, you can build a reliable and efficient monitoring system with Prometheus, enabling you to gain valuable insights into your infrastructure's performance and health.

Chapter 6

Conclusion

In the journey through the world of Prometheus, we've delved into the heart of modern monitoring and observability. Prometheus, a titan of innovation in the realm of open-source software, has empowered us to peer deeply into the intricate workings of our digital landscapes. As we close this chapter, we reflect on the profound impact Prometheus has made, transforming the way we monitor, troubleshoot, and optimize our systems.

From the inception of a simple idea—capturing and querying metrics—to the evolution of a full-fledged ecosystem, Prometheus has become synonymous with resilience and insight. Its powerful querying language, PromQL, is a conduit that allows us to converse with our data, to query not just for numbers but for understanding. Prometheus has taught us to be proactive rather than reactive, to anticipate issues and trends, and to ensure the seamless performance of our applications.

Yet, as Prometheus has guided us on a journey of enlightenment, it has also reminded us of the importance of balance. It is not a panacea, but rather a tool wielded by skilled hands. The path to mastery requires not only technical prowess but a deep understanding of our systems and the insight to transform data into actionable wisdom.

As we bid farewell to this exploration of Prometheus, let us carry forward the torch it has ignited. Let us continue to champion the principles of observability, to nurture the bond between humans and machines, and to embrace the relentless pursuit of improvement. Just as Prometheus stole fire from the gods to illuminate the world, may the knowledge we've gained kindle a brighter future for the landscapes we traverse.

Chapter 7

About the author

Hello! I am Alexis Carbillet, a graduate from the Artificial Intelligence program at IMT Mines Alès who is passionate about data. I am currently working at OVHcloud and on different side projects such as my blogs. You can find them on my website

Thank you for reading this book. I sincerely hope it was usefull to you.

Else you can simply use this QR code: